

System Modules – Responsibilities and Details

The **car rental system** now includes three user roles (Customer, Owner, Admin). We break the system into modules, each with clear responsibilities:

- **Authentication Module** – Manages user registration, login, and session handling for all roles (customers and owners; admin accounts are created by the system).
- **Owner Management Module** – Handles owner-specific functions: owner registration, profile management, and car listings.
- **Car Listing & Search Module** – Owners add cars, customers search/filter available cars, and detailed car info is shown.
- **Booking Module** – Customers request rentals; owners approve or reject; admin oversees bookings.
- **Payment/Payout Module** – Customers pay for bookings; platform collects commission and disburses owner payouts.
- **Chat/Communication Module** – Enables messaging between customers and owners (and admin if needed).
- **Rating & Feedback Module** – After each rental, customers and owners rate each other.
- **Admin Module** – Admin manages users/owners/listings, handles verification and payouts, and monitors the system.

Each module involves UI pages (React components), backend APIs/services, database tables, and end-to-end workflows. We detail these below.

Authentication Module

Responsibilities:

- Register new users (customers or owners) and log them in.
- Store credentials securely and manage sessions.
- Provide role-based access control (OWNER vs CUSTOMER vs ADMIN).

Frontend:

- **Register Page:** Separate flows or a single form with role selection. Component (e.g. `<RegisterForm>`) collects name, email, password, and account type (customer/owner).
- **Login Page:** `<LoginForm>` for email/password. Redirect to role-specific dashboard upon success.
- **Forgot Password:** (Optional) page to reset credentials.

Backend APIs/Services:

- `POST /api/auth/register` – Accepts `{name, email, password, role}`, creates a new user in `Users` table (with role “CUSTOMER” or “OWNER”). For owners, additional verification (e.g. KYC) can be initiated here or by admin.
- `POST /api/auth/login` – Validates credentials, returns a session or token.
- `GET /api/auth/me` – Returns current user info (protected).

- **Security:** Protect routes based on role (e.g. only ADMIN can access `/api/admin/*`, only OWNER can access `/api/owners/*`). Sessions or Spring Security handle this ¹.

Database Tables:

- **Users:** (user_id PK, name, email UNIQUE, password_hash, role ENUM('CUSTOMER', 'OWNER', 'ADMIN'), contact_info, etc.).
- **Owners (optional):** If extra owner info is needed (e.g. payout details), a separate `Owners` table (owner_id PK, user_id FK→Users, bank_account, verified BOOLEAN).

Workflow:

1. User fills registration form (including selecting “Owner” or “Customer”) and submits to `/api/auth/register`.
2. The server hashes the password, creates a `Users` record, and if role=OWNER, creates an `Owners` entry (or flags user as owner).
3. On login (`/api/auth/login`), credentials are checked, and a session cookie or JWT is returned.
4. The frontend stores the session and redirects the user to the appropriate dashboard (owner vs. customer view).

Owner Management Module

Responsibilities:

- Enable owners to manage their accounts and list vehicles.
- Owners must authenticate and be verified by admin before listing cars.
- Owners handle booking requests for their cars.

Frontend:

- **Owner Dashboard:** After login, `<OwnerDashboard>` shows summary (pending bookings, earnings, etc.).
- **Owner Profile Page:** `<OwnerProfile>` to view/update personal info (contact, documents).
- **My Cars Page:** `<MyCarsList>` lists all cars owned; button to add a new car.
- **Add/Edit Car Page:** `<CarForm>` where owner enters vehicle details (make, model, year, price, location, photos, availability schedule).
- **Booking Requests Page:** `<OwnerBookings>` lists booking requests on the owner’s cars (pending approval).
- **Payout/Earnings Page:** `<OwnerPayouts>` shows past rentals, amounts due, and payout history.

Backend APIs/Services:

- `POST /api/owners/register` (or combined with auth) – Similar to `/auth/register`, but may capture owner-specific data (e.g. documentation). Admin will verify this.
- `GET /api/owners/{ownerId}/profile` – Owner’s profile info.
- `PUT /api/owners/{ownerId}/profile` – Update owner info (contact, payout details).
- `POST /api/owners/{ownerId}/cars` – Add a new car listing. Body includes car details.
- `GET /api/owners/{ownerId}/cars` – Get list of owner’s cars.
- `PUT /api/owners/{ownerId}/cars/{carId}` – Update car info or availability.
- `DELETE /api/owners/{ownerId}/cars/{carId}` – Remove a listing.
- `GET /api/owners/{ownerId}/bookings` – Get bookings for this owner’s cars (filter by status).
- `POST /api/owners/{ownerId}/bookings/{bookingId}/approve` – Owner approves a pending

booking ² .

- `POST /api/owners/{ownerId}/bookings/{bookingId}/reject` – Owner rejects a request.
- `GET /api/owners/{ownerId}/earnings` – Summary of confirmed rentals and earnings.

Security:

- Endpoints under `/api/owners/*` require an authenticated user with role OWNER. For example, the approve/reject endpoints check that `ownerId` matches the authenticated user's ID.

Database Tables:

- **Cars:** (`car_id` PK, `owner_id` FK→Users(or Owners), `category_id`, `make`, `model`, `year`, `price_per_day`, `location`, `availability_status`, ...). Each car belongs to an owner.
- **CarCategory:** (`category_id` PK, `name`, `seats`, `luggage`, `price_multiplier`, ...). (Referenced by Cars.)
- The `Cars` table fields include things like `availability_flag` or `schedule` to indicate when the car can be rented.

Workflow:

1. **Listing a car:** Owner fills the Add Car form and submits it. The frontend calls `POST /api/owners/{id}/cars`. The CarService creates a `Cars` record with `owner_id` set.
2. **Approval:** Optionally, admin automatically approves new car listings, making them visible.
3. **Receiving Booking Requests:** When a customer books the car, a new booking record appears with status "PendingOwner". It also shows up in the Owner's Bookings page.
4. **Responding to Requests:** The owner clicks "Approve" on a booking. Frontend calls `POST /api/owners/{id}/bookings/{bookingId}/approve`. The service sets the booking status to "OwnerApproved", triggering the payment step.
5. **Payout:** After rentals complete, the earnings are tallied. The Admin later processes payouts (see Payment module).

Source: Owner features commonly include managing vehicle listings and booking requests ³ ² , with dashboards listing cars and reservations as noted in platform guides ⁴ .

Car Listing & Search Module

Responsibilities:

- Store and organize car listings by owners.
- Allow customers to search/filter available cars by location, date, category, price, etc.
- Display detailed car information to customers.

Frontend:

- **Search Page:** `<CarSearch>` component with fields for location, date range, filters.
- **Car Listings Page:** `<CarList>` showing cards or list items of matching cars (with summary info like photo, model, price).
- **Car Details Page:** `<CarDetails>` shows full information (owner, specs, images, pricing) and includes a "Request Booking" or "Book Now" button.
- **Filters Component:** Reusable filters (min/max price, category dropdown, etc.).

Backend APIs/Services:

- `GET /api/cars` - Returns cars matching query params (e.g. `?location=X&fromDate=YYYY&toDate=YYYY&category=SUV`). The service checks that cars are not already booked in that range.
- `GET /api/cars/{carId}` - Get details of one car (with owner info).
- `GET /api/car-categories` - List of categories.
- (Admin endpoints to add categories can exist, or it's seeded.)

Security:

- These endpoints are typically public (no login required) or available to any authenticated role. Admin & owner can also use these for management.

Database Tables:

- **Cars:** (as above) contains all key fields.
- **CarCategories:** To classify cars (e.g. Economy, Sedan, SUV). Columns like `(category_id, name, seats, luggage, base_rate)`.
- **Availability/Reservations:** Rather than a separate table, availability is derived: a car is available if no booking overlaps the requested dates.

Workflow:

1. **Search:** Customer enters criteria, React calls `GET /api/cars`. The query goes to CarController, which invokes CarService to query the DB (joining Cars and Bookings to filter out conflicts).
2. **Display Results:** The API returns a JSON list of available cars; the `<CarList>` component renders them.
3. **View Details:** On clicking a car, the app calls `GET /api/cars/{carId}` to fetch full details, which `<CarDetails>` then shows.

Peer platforms typically provide comprehensive filtering and sorting, e.g. by price or features, to help renters find suitable vehicles. According to a Turo-like feature table, owners can “manage listed vehicles” while customers “browse available cars” ⁴.

Booking Module

Responsibilities:

- Manage rental requests from customers and confirmations by owners.
- Track booking statuses (e.g. Pending, Approved, Confirmed, Completed, Cancelled).

Frontend:

- **Booking Form:** On `<CarDetails>`, `<BookingForm>` lets the customer pick dates and submit a booking request.
- **My Bookings Page (Customer):** `<CustomerBookings>` shows the customer's booking history and statuses.
- **Owner Booking Requests (Owner):** `<OwnerBookings>` (as above) lists requests with action buttons.
- **Booking Confirmation Page:** `<BookingSummary>` shows details once booking is confirmed.

Backend APIs/Services:

- `POST /api/bookings` - Customer requests a booking. Body includes `{car_id, from_date,`

`to_date, pickup_location, dropoff_location}`. The service: 1. Checks if the car is still available for those dates. 2. Creates a new record in `Bookings` with `user_id = customer`, `car_id`, and `status = 'PendingOwner'`.

- `GET /api/bookings` - Returns bookings for the current user (customer or owner based on role).
- `GET /api/bookings/{bookingId}` - Booking details.
- `POST /api/bookings/{bookingId}/cancel` - Customer or owner can cancel a pending booking (status set to Cancelled).
- **Owner Approve/Reject:** (As above: `/api/owners/{ownerId}/bookings/{bookingId}/approve` & `/reject`) sets status accordingly. Once approved, the status becomes "OwnerApproved".
- `POST /api/bookings/{bookingId}/confirm-payment` - (called after payment) sets status to "Confirmed".

- Security:

- Customers can create/cancel their own bookings. Owners can approve/reject bookings only for their cars. Admin can view or cancel any booking. Controllers enforce these checks (e.g. verify owner owns the car in the booking).

Database Tables:

- **Bookings:** (`booking_id` PK, `car_id` FK-Cars, `user_id` FK-Users (customer), `from_datetime`, `to_datetime`, `pickup_loc`, `dropoff_loc`, `status`, `total_amount`, `created_at`).
- **(Optional) BookingExtras:** If adding insurance or add-ons, e.g. (`booking_id` FK, `insurance_opted`, etc.).

Workflow:

1. **Request:** Customer fills booking form; React calls `POST /api/bookings`. Backend `BookingsService` checks availability and saves the booking with status "PendingOwner".
2. **Owner Notification:** The new request appears in the owner's bookings list. Optionally, the system notifies the owner via email or in-app notification.
3. **Owner Decision:** Owner calls `/approve` or `/reject`. If approved, the service updates status to "OwnerApproved"; if rejected, "Cancelled".
4. **Payment:** When status is "OwnerApproved", the customer is prompted to pay (`Payment Module`).
5. **Finalizing:** After payment, `POST /api/bookings/{id}/confirm-payment` sets status "Confirmed". Later, on car return, an endpoint `/return` sets status "Completed" and triggers feedback.

Industry practice notes that owners can "accept or decline requests" for their vehicles ². Thus, the two-step approval (request → approve → payment) is built into the booking flow.

Payment and Payout Module

Responsibilities:

- Process customer payments securely.
- Calculate and distribute owner payouts (after commission).
- Track financial transactions and status.

Frontend:

- **Payment Page:** `<PaymentForm>` where customer enters payment info (card details, or selects saved

method).

- **Payment Success Page:** `<PaymentSuccess>` shows receipt or booking confirmation.

- **Owner Payout Page:** `<Payouts>` lists amounts owed and paid to the owner.

- **Admin Payment Dashboard:** `<AdminPayments>` shows all payments and payouts.

Backend APIs/Services:

- `POST /api/payments` - Handles actual charging (using a gateway). Body includes `{booking_id, payment_method, amount}`. On success, the service: - Creates a `Payments` record (with `booking_id`, `amount`, `timestamp`).

- Updates the associated booking's status to "Confirmed".

- Calculates the commission split: e.g. `owner_amount = amount * 0.8`, `admin_amount = amount * 0.2`.

- Creates a pending `Payouts` record: `(owner_id, booking_id, owner_amount, status='Pending')`.

- `GET /api/payouts/{ownerId}` - For an owner to view their pending and completed payouts.

- `POST /api/admin/payouts/{payoutId}/complete` - Admin marks a payout as completed (i.e. money transferred to owner).

- Security:

- Payment endpoint requires customer authentication. Admin/payout endpoints require ADMIN role. Owners see only their payouts.

Database Tables:

- **Payments:** (`payment_id` PK, `booking_id` FK→Bookings, `amount`, `payment_date`, `method`, `status`).

- **Payouts:** (`payout_id` PK, `owner_id` FK→Owners (or Users), `booking_id` FK, `amount`, `commission`, `payout_date`, `status`). Stores the owner's share, admin commission, and when paid.

Workflow:

1. **Charge Customer:** After owner approval, React calls `POST /api/payments`. The `PaymentService` integrates with a gateway (Stripe, etc.) to charge the card.

2. **Record Transaction:** On gateway success, a `Payments` entry is created, and the booking status is updated to "Confirmed".

3. **Compute Split:** The service calculates the commission (e.g. 20%) and owner's share. It inserts a `Payouts` record marked Pending.

4. **Notify Owner:** Optionally, notify the owner of the confirmed booking and upcoming payout.

5. **Payout Processing:** Periodically, the admin reviews pending payouts and calls the API to mark them as paid. The system transfers funds to the owner's account (outside the app) and updates the record to "Paid".

Source: This split model matches marketplace platforms where "admin transfers money to the car owner after keeping commission" ⁵. For example, Turo's model includes booking fees and host earnings, which our payout module mimics.

Chat / Communication Module

Responsibilities:

- Enable real-time messaging between customers and owners (and admins).
- Store chat history for transparency and support.

Frontend:

- **Chat Window:** `<ChatWindow>` component for 1-on-1 messaging. It appears on relevant pages (e.g. on the booking or car details page, a "Message Owner" button opens it).
- **Chat List:** `<ChatList>` showing active chat threads (for an owner or user).
- Reusable message components (bubble UI) with styling via CSS or a UI library.

Backend APIs/Services:

- **WebSocket/Stomp:** Endpoints for real-time. For example, `@MessageMapping("/chat")` on server. Clients subscribe to `/topic/{channel}`.
- **REST Fallback:** `POST /api/chat/{bookingId}/message` - Stores a message in `ChatMessages`. (Useful to load history or if WebSockets fail.)
- **Security:** Only allow sending/receiving in threads where the user is a participant (owner or customer of the booking).

Database Tables:

- **ChatMessages:** (message_id PK, booking_id FK-Bookings, sender_id FK-Users, recipient_id FK-Users, content TEXT, timestamp).
- Storing `booking_id` links chats to a specific rental and implicitly ties together customer/owner.

Workflow:

1. **Initiating Chat:** When a booking is created, a chat thread (identified by `booking_id`) is opened. Owner and customer subscribe to that thread via WebSocket.
2. **Real-Time Messaging:** A user sends a message; frontend sends via STOMP. The server `ChatController` broadcasts it to the other participant's subscription and writes it to `ChatMessages`.
3. **Message Delivery:** The recipient's React `<ChatWindow>` receives the message event and updates the chat UI.
4. **History:** On loading the chat component, React can `GET /api/chat/{bookingId}` to retrieve past messages from `ChatMessages`.

According to peer-to-peer car rental practices, these systems provide communication channels so "car owners and renters can exchange messages" about bookings ⁶.

Rating & Feedback Module

Responsibilities:

- Collect post-rental reviews and ratings from both sides.
- Display average ratings on user and car profiles.

Frontend:

- **Rating Page:** After a rental is completed, `<ReviewForm>` prompts the user to rate the car/owner (e.g. 1-

5 stars, comments).

- **Reviews Display:** `<ReviewList>` on the car details page or owner profile shows past reviews.
- **Owner Reviews:** Similarly, owners can submit ratings of the renter.

Backend APIs/Services:

- `POST /api/reviews` - Submit a review. Body includes `{booking_id, rating_for_user_id, rating_value, comments}`. The server checks that the reviewer is part of the booking.
- `GET /api/cars/{carId}/reviews` - List reviews left for that car.
- `GET /api/users/{userId}/reviews` - List reviews for an owner or customer profile.
- **Security:** Only authenticated participants can leave reviews for a completed booking.

Database Tables:

- **Reviews:** (review_id PK, booking_id FK→Bookings, reviewer_id FK→Users, subject_id FK→Users, rating INT, comments TEXT, review_date).
- A review could refer to either a car or a user. For simplicity, we link to `subject_id` (the user being reviewed) and also use `booking_id` to attach it to a rental.

Workflow:

1. **Completion:** When a booking's status changes to "Completed" (car returned), the frontend offers a review form to the user and owner.
2. **Submitting:** User posts review via `POST /api/reviews`. The service saves it and updates aggregate rating for the car or owner if desired.
3. **Displaying:** The reviews are shown on profiles. For example, the Car Details page might list past reviews of that car (by accessing `/cars/{id}/reviews`).

Platforms emphasize this feedback loop: owners and renters both "can provide ratings and reviews" to build trust
7.

Admin Module

Responsibilities:

- Oversee the entire system.
- Approve owners and car listings (verification).
- Manage commissions and payouts.
- Monitor bookings, payments, and user activity.
- Provide support and handle disputes.

Frontend:

- **Admin Dashboard:** `<AdminDashboard>` with summaries (total users, active rentals, revenue).
- **User/Owner Management:** `<AdminUsers>` lists all customers and owners; admin can deactivate accounts or verify documents.
- **Car Listings Management:** `<AdminCars>` shows all cars on the platform; admin can remove inappropriate listings.
- **Booking Management:** `<AdminBookings>` view all bookings, intervene (cancel, refund) if needed.
- **Financials:** `<AdminPayments>` displays all payments collected and payouts made; admin triggers

payouts to owners.

- **Settings:** Pages to configure commission rates and policies.

Backend APIs/Services:

- GET /api/admin/users – List all users/owners.
- PUT /api/admin/users/{id}/verify – Verify or ban a user/owner.
- GET /api/admin/cars – List all car listings.
- DELETE /api/admin/cars/{carId} – Remove a listing.
- GET /api/admin/bookings – List all bookings.
- GET /api/admin/payments – List all payments.
- POST /api/admin/payouts/{id}/complete – Mark an owner's payout as sent.

Security:

- Endpoints under /api/admin/* require ADMIN role. Controllers enforce only admins can access these.

Database Tables:

Admin uses the same tables (Users, Cars, Bookings, Payments, Payouts). There is no separate table needed for admin functions, though an AuditLog table could record admin actions if desired.

Workflow:

1. **Verifying Owners:** When an owner registers, their status may be "Unverified." Admin reviews their documents and calls PUT /api/admin/users/{ownerId}/verify to activate the account.
2. **Handling Disputes:** If disputes arise, admin can change booking status or issue refunds.
3. **Payout Execution:** Admin approves pending payouts via the API. After payment, the Payouts record is updated to "Paid."
4. **Reporting:** Admin might download reports of activity or revenue (not explicitly coded, but could be derived via queries).

The admin truly acts as an intermediary: the architecture outlines that Admin "will act as a middle man between Car Owner and Customer" ⁸, handling verification and funds transfer.

Inter-Module Workflows and Data Flow

• End-to-End Booking Example:

- Customer searches for cars (GET /api/cars).
- Selects a car and submits booking form (POST /api/bookings). Booking created with status Pending.
- Owner approves it (POST /api/owners/{id}/bookings/{bookingId}/approve). Status = OwnerApproved.
- Customer pays (POST /api/payments). Status = Confirmed; Payments and Payouts records created.
- At return, customer/owner signals /api/bookings/{id}/return . Status = Completed. Feedback can be given.
- Admin reviews and processes payment to owner via /api/admin/payouts/{payoutId}/complete .

- **Data Flow:** Frontend ↔ REST API ↔ Spring services ↔ Hibernate Repositories ↔ MySQL tables. For example, a booking request triggers the BookingController, which uses BookingService to insert into the `Bookings` table. The car availability check queries `Bookings` and `Cars` tables. Citations on standard Spring layers apply ¹ ⁹.
- **Notifications:** While not a separate module, each step triggers notifications (e.g. email or in-app) – e.g., booking confirmations, status changes, payment receipts. These can be implemented via message queues or background services.

Each module above works with clear inputs/outputs. For instance, the **Car Listing** module's database tables (`Cars`, `CarCategories`) feed the **Search** API. The **Booking** and **Payment** modules share the `Bookings` and `Payments` tables. This design ensures that responsibilities are separated but integrated in data flow.

Sources: We used platform best practices and industry examples to inform each module. For example, feature breakdowns for customer/admin/owner apps list functions like owner vehicle listing and booking request management ⁴. Peer-to-peer rental guides describe owners creating detailed vehicle listings ³ and approving bookings ². We have tailored our module structure to ensure thorough coverage for each role, with citations illustrating key features (e.g. owner approvals ², admin commission handling ⁵, and communication ⁶).

¹ Spring Boot - Architecture - GeeksforGeeks

<https://www.geeksforgeeks.org/springboot/spring-boot-architecture/>

² ³ ⁶ ⁷ Blog - All Ride Apps

<https://www.allrideapps.com/peer-to-peer-car-rental-software-companies/>

⁴ Step-by-step Process to Build Car Rental Like Turo

<https://www.valueappz.com/blog/process-to-build-car-rental-app-like-turo>

⁵ ⁸ GitHub - farazkhawaja/Car-rental-system: two-way car renting system(Semester Project) · GitHub

<https://github.com/farazkhawaja/Car-rental-system>

⁹ reactjs - What are the architecture tiers of a Spring Boot + React + MySQL application? - Stack Overflow

<https://stackoverflow.com/questions/76542321/what-are-the-architecture-tiers-of-a-spring-boot-react-mysql-application>